DSA0501 – QUERY PROCESSING FOR DATA SCIENCE

ASSIGNMENT 1 – CO1

CityCare Health Network: Designing, Implementing and Evaluating a Multi-Format Data-Wrangling Pipeline

Name: Answar Vijes

Register Number: 192324028

Nature: Individual Assignment

Total Marks: 50

Submitted as an academic solution based on the supplied assignment brief and its seed records.

Declaration and scope

This report answers Tasks 1–12 and the Day-4 Design Challenge using the CityCare scenario and sample records in the supplied assignment. The sample contains only a small number of records; therefore, sample-derived observations are distinguished from proposed targets and test procedures. No claim is made that a 200-row/100-report/100-dispense generator, a second-tool experiment, or a 20,000-line Day-4 benchmark was executed in this report. Those measurements must be produced by running the supplied scripts on the student's machine. This distinction prevents invented experimental evidence.

Contents

• Task 1 – Problem investigation

• Task 2 – End-to-end workflow

• Task 3 – Wrangling tasks and rules

• Task 4 – Tool comparison

• Task 5 – Dataset generation and profiling

• Task 6 – Machine-readable data

• Task 7 – CSV processing

• Task 8 – JSON processing

• Task 9 – XML processing

• Task 10 – Database schema

• Task 11 – Integration and conversion
• Task 12 – Testing and evaluation
• Day-4 D1–D4 – Design challenge
• Appendix – Core implementation patterns and run checklist

Task 1: Problem Investigation – The Cost of Unwrangled Data

1.1 Meaning and importance of data wrangling

Data wrangling is the repeatable process of discovering, structuring, cleaning, standardising, enriching, validating and publishing raw data so that it is reliable for a defined purpose. At CityCare, it means bringing OPD CSV visits, nested laboratory JSON and pharmacy XML into a consistent patient-centred dataset. It includes more than correcting spelling: identifiers must be reconciled, dates and amounts parsed, nested tests and medicine items reshaped, duplicates controlled, missing values flagged and relationships checked before records enter the central database. Data cleaning is one part of wrangling and focuses mainly on defects such as blanks, duplicates and invalid values. ETL (extract, transform, load) is an implementation pattern for moving data between systems; wrangling supplies much of the discovery and transformation logic within it. Data analysis interprets already prepared data to answer questions. Wrangling is a prerequisite because analysis on mismatched IDs, incorrect totals or ambiguous dates can produce misleading patient counts, unsafe clinical conclusions and incorrect bills. A documented pipeline also makes corrections repeatable, traceable and auditable.

1.2 Defect and consequence analysis

Defect	Source	Example from sample	Probable cause	Consequence	Severity
Duplicate visit	A	P1001 row appears twice with same date, department and fee	Manual re-registration/export duplication	Inflated visit and revenue counts	High
Mixed date formats	A	14/03/1985; 2026-10-03; 03-10-2026	Different data-entry conventions	Wrong visit or age calculations	High
Inconsistent gender codes	A	M, Female, m, F, Male	No controlled vocabulary	Incorrect demographic reports	Medium
Missing phone	A	P1002 phone is blank	Not captured at registration	Follow-up contact impossible	Medium
Missing patient name	A	P1005 full_name is blank	Incomplete registration	Identity matching becomes unsafe	High
Currency as text	A	"1,200.50" and "■ 1500"	Locale and manual formatting	Fees omitted or mis-summed	High
Patient ID style mismatch	B	1001 versus P1001	Independent system identifiers	Lab linked to wrong/no patient	High
Mixed lab result type	B	112 as string; 13.2 numeric; pending text	LIS status/value mixed in one field	Pending may be mistaken for numeric zero	High
Optional blocks/ranges	B	Second report has no ordered_by; ref_range null	Optional JSON objects	Flattening code may crash or misclassify	Medium
Date style inconsistency	C	04.10.2026 and 2026-10-05	Different export settings	Incorrect dispensing chronology	High
Non-numeric quantity	C	qty="two"	Manual or system validation failure	Cannot calculate prescription total	High
Missing total	C	RX-88122 has no total element	Incomplete export	Billing total cannot be reconciled	High
Total mismatch	C	RX-88121 items: 15×4.5 + 10×2.25 = 90.00	Source total may be stale or calculated differently	Billing error if unchecked	High

The RX-88121 total is internally consistent: 67.50 + 22.50 = INR 90.00. RX-88122 cannot be recomputed because its quantity is the non-numeric word “two” and its total is absent.

1.3 Stakeholders

Stakeholder	Needs	If defects remain
Billing officer	Unique visits, numeric fees, reconciled prescription totals and traceable adjustments	Duplicate or missing charges; incorrect invoices and revenue totals
Treating physician	Correct patient linkage, timestamped lab values, units, ranges and explicit pending status	Wrong-patient results or pending tests interpreted as completed results
Quality auditor / state reporting team	Unique patient counts, consistent demographics, validated dates and audit logs	Unreliable quarterly statistics and audit findings
Registration staff	Clear validation feedback and a controlled patient lookup	Repeated duplicate records and avoidable re-entry

1.4 Measurable objectives

• At least 99% of required visit dates parse to ISO 8601; all unparseable dates are quarantined rather than guessed.

• At least 99% of non-orphan source records resolve to one canonical patient ID; ambiguous matches are not auto-merged.

• 100% of loaded prescription totals either match recomputed item totals within INR 0.01 or carry an explicit exception status.

1.5 Profiling script for the seed CSV

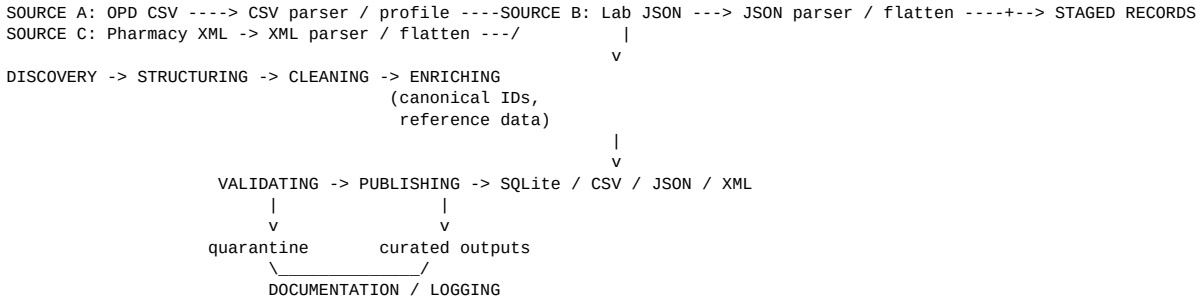
```
import csv
from collections import defaultdict

path = "source_a.csv"
missing = defaultdict(int)
categories = defaultdict(set)
with open(path, newline="", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for row in reader:
        for key, value in row.items():
            value = (value or "").strip()
            if not value or value.upper() in {"NA", "N/A", "NULL"}:
                missing[key] += 1
            if key in {"gender", "department"} and value:
                categories[key].add(value)
print("missing:", dict(missing))
print("distinct categorical spellings:",
      {k: sorted(v) for k, v in categories.items()})
```

For the six supplied CSV rows, visible missing values are: full_name = 1, phone = 1, dob = 1 and fee_paid = 1 if “NA” is treated as missing. Gender spellings include M, Female, m, F and Male (five distinct raw spellings); department spellings include Cardiology, ENT, Orthopaedics, Paediatrics and General Medicine (five). The repeated P1001 row is a duplicate candidate. The script reports actual values when run against the saved seed file.

Task 2: Designing the End-to-End Wrangling Workflow

2.1 Pipeline diagram



The three formats are first inspected and parsed independently because their syntax and failure modes differ. They merge only after each source has been converted into staged records with canonical field names and types. Enrichment then resolves IDs and adds reference data. Validation occurs before publication so that invalid or ambiguous records cannot silently enter the trusted database.

2.2 Stage table

Stage	Inputs and operations	Output	Quality gate
Discovery	Inventory files, encoding, headers, JSON key paths, XML elements, row counts and date patterns	Source profile and data dictionary	All expected files accounted for; unknown structures flagged
Structuring	Parse CSV/JSON/XML; flatten tests and dispensing items; map fields to canonical names	Typed staged records	Parser errors isolated; required structural fields present
Cleaning	Trim strings, parse dates/amounts, standardise gender/name/phone, detect duplicates	Normalised records plus rejects	No silent coercion; every rejected row has a reason
Enriching	Canonicalise patient IDs; map doctors/departments; attach visit context	Linked records	Unique deterministic match or explicit orphan/ambiguous status
Validating	Check domains, keys, date logic, quantities, totals and foreign keys	Validated records and exception report	All critical constraints pass or record quarantined
Publishing	Load SQLite transactionally; export consumer-specific CSV/JSON/XML	Curated database and files	Row reconciliation and referential integrity pass
Documentation/logging	Record run ID, file hash, code version, counts, errors and decisions	Audit trail	Logs complete; rerun is idempotent

2.3 Storage layout and raw-data policy

```
citycare_project/
  config.py
  data/
    raw/source_a/ source_b/ source_c/
    staged/source_a/ source_b/ source_c/
    curated/csv/ curated/json/ curated/xml/
    rejected/
  logs/
  reports/
  src/
  tests/
```

Raw files are immutable evidence of what the source system delivered. Editing them in place destroys provenance, makes it impossible to reproduce a previous result and may conceal who changed a patient record. Every run reads raw inputs and writes new staged/curated outputs. Store a cryptographic hash, file name, received timestamp and run ID for each input.

2.4 Audit, errors and safe reruns

• For each rejected record log: run ID, source file, record number/ID, timestamp, error code, field, original value (masked where sensitive), reason, severity and action taken.

• A missing file stops only the dependent source branch and raises an alert; the pipeline must not publish an apparently complete dataset. A malformed file is quarantined with parser line/column details; other independent sources may continue if the run is explicitly marked partial.

• Write outputs to a run-specific temporary directory, validate them, then atomically promote them. Use database transactions and unique keys/upserts. A repeated run with the same input hash and pipeline version must not create duplicate visits.

• Never put full patient names, phone numbers or lab values in general-purpose logs. Restrict access, encrypt storage and retain logs according to policy.

2.5 Program map and scaffold

Program	Responsibility
setup_project.py / config.py	Create folders and centralise paths, date formats, canonical columns and thresholds
utils.py	Reusable parsing, normalisation and validation functions
generate_data.py / profile.py	Generate reproducible dirty data and measure injected defects
csv_wrangler.py	Read, clean, deduplicate, reconcile and export OPD visits
json_wrangler.py	Inspect nested lab JSON, flatten tests and classify results
xml_wrangler.py	Parse pharmacy XML, recompute totals and export item rows
schema.py	Create constrained SQLite schema and demonstrate constraints
integrate.py / convert.py	Canonical ID matching, database loading and format exports
test_pipeline.py / quality_report.py	Unit/integration/negative tests and before/after metrics
json_lines_wrangler.py	Stream the Day-4 JSON Lines source

```
# setup_project.py
from pathlib import Path
ROOT = Path(__file__).resolve().parent
for folder in ("data/raw/source_a", "data/raw/source_b",
               "data/raw/source_c", "data/staged",
               "data/curated", "data/rejected", "logs",
               "reports", "src", "tests"):
    (ROOT / folder).mkdir(parents=True, exist_ok=True)
print("Project folders ready:", ROOT)
```

```
# config.py
from pathlib import Path
ROOT = Path(__file__).resolve().parent
RAW = ROOT / "data" / "raw"
STAGED = ROOT / "data" / "staged"
CURATED = ROOT / "data" / "curated"
DATE_FORMATS = ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y",
                "%Y/%m/%d", "%d.%m.%Y", "%d-%b-%Y")
CANONICAL_VISIT_COLUMNS = (
    "patient_id", "full_name", "gender", "dob", "phone",
    "visit_date", "department", "fee_amount")
THRESHOLDS = {"date_validity": 0.99, "id_match": 0.99,
              "rx_total_match": 1.00}
```

```
# environment_check.py
import sys, csv, json, xml.etree.ElementTree as ET
import sqlite3, datetime, re, logging, unittest
print("Python:", sys.version)
for module in (csv, json, ET, sqlite3, datetime, re, logging, unittest):
    print(module.__name__, "OK")
```

Task 3: Identifying Data-Wrangling Tasks and Rules

3.1 Defect-to-task matrix

Defect	Task type	Rule
Mixed dates	Parsing / standardisation	Parse only declared formats; output YYYY-MM-DD; ambiguous values use documented locale policy or quarantine
Gender spellings	Standardisation	M/Male/m -> M; F/Female/f -> F; other values -> UNKNOWN + flag
Currency strings	Type conversion	Remove INR/■ and grouping commas only after validating numeric pattern; Decimal to 2 places
Duplicate visit	De-duplication	Key=(canonical patient_id, visit_date, department); keep first source occurrence; log duplicate
Blank name	Missing-value handling	Reject patient master creation; do not fabricate a name
Blank phone	Missing-value handling	Retain null with CONTACT_MISSING flag; phone is not a patient key
1001 / P1001	Merging	Strip prefixes P/PT and normalise numeric suffix to canonical integer string for legacy IDs
Lab pending	Type conversion / validation	Store value_num=NULL and result_state=PENDING; never convert to zero
Null reference range	Validation	Flag range unavailable; classify result UNKNOWN rather than infer a clinical range
Missing ordered_by	Missing-value handling	Retain report with doctor_id=NULL and flag; doctor is optional in sample
XML qty=two	Type conversion	Reject item quantity; quarantine dispense until corrected or verified
Missing XML total	Validation / enrichment	Recompute from valid items; retain computed total with COMPUTED_TOTAL flag
Nested tests/items	Flattening	One output row per test or drug item; preserve report/dispense parent ID

3.2 Ambiguous cases and evidence

Case	Decision	Evidence and justification
03-10-2026	Interpret as 3 October 2026 for this sample	Assignment describes the OPD example alongside 2026-10-03 and the lab report on 2026-10-03; retain the same-day interpretation. For production, source-specific locale must be confirmed.
14/03/1985	DD/MM/YYYY	14 cannot be a month, so this format is unambiguous.
Blank phone	Retain null and flag	Phone is not a declared primary key; rejecting the whole visit would lose clinical data. Do not invent a number.
Duplicate P1001 visit	Keep first occurrence	Same ID, name, DOB, visit date, department and fee are repeated exactly.
Fee NA	Null fee + billing exception	No defensible numeric amount can be inferred; never replace with zero because it changes revenue.
Lab value pending	Null numeric result; status pending	Text is a workflow state, not a measured numeric value.
XML qty two	Reject item / dispense exception	No numeric quantity can be calculated safely.
RX total missing	Recompute only if all item quantities/prices valid	For RX-88122 qty is invalid, so no reliable total can be calculated.

3.3 Missing-data policy

Field	Policy	Risk / rationale
patient_id	Reject if absent or unresolved	Wrong-patient linkage is a high clinical risk
full_name	Reject master-patient creation; retain source row in reject queue	Name is NOT NULL in target schema and needed for identity review
gender	Retain as UNKNOWN with flag	Do not infer sensitive demographic data

Field	Policy	Risk / rationale
DOB	Retain null with flag; exclude from age calculations	Avoid fabricated age or date
phone	Retain null with contact flag	Operational follow-up may fail, but visit remains useful
visit_date	Reject if unparseable	Chronology and matching cannot be trusted
fee_paid	Retain null and billing exception	Do not silently set NA to zero
lab result value	Pending => null + state; malformed => exception	A missing result is not a zero result
ref_range	Retain null; flag UNKNOWN	No clinical threshold should be invented
prescription quantity	Reject invalid item	Incorrect quantity changes dispensing and billing totals
prescription total	Compute if all items valid; otherwise exception	No unsupported imputation

3.4 Utility functions and demonstrations

```
# utils.py
from datetime import datetime
from decimal import Decimal, InvalidOperation
import re

def normalise_gender(value):
    v = str(value or "").strip().lower()
    if v in {"m", "male"}: return "M"
    if v in {"f", "female"}: return "F"
    return None

def parse_date(value):
    if not value or not str(value).strip(): return None
    s = str(value).strip()
    formats = ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y",
               "%Y/%m/%d", "%d.%m.%Y", "%d-%b-%Y")
    for fmt in formats:
        try: return datetime.strptime(s, fmt).date().isoformat()
        except ValueError: pass
    return None

def clean_phone(value):
    digits = re.sub(r"\\D", "", str(value or ""))
    return digits if 10 <= len(digits) <= 15 else None

def parse_amount(value):
    if value is None: return None
    s = str(value).replace("■", "").replace("INR", "").strip()
    s = s.replace(",", "")
    try: return Decimal(s).quantize(Decimal("0.01"))
    except (InvalidOperation, ValueError): return None

def standardise_name(value):
    if not value or not str(value).strip(): return None
    return " ".join(str(value).strip().split()).title()
```

Input	Output	Rule
M / Male / m	M	Case-insensitive male mapping
F / Female	F	Case-insensitive female mapping
14/03/1985	1985-03-14	DD/MM/YYYY
03-10-2026	2026-10-03	DD-MM-YYYY under declared source policy
2026/10/05	2026-10-05	YYYY/MM/DD
98400 56789	9840056789	Remove separators; validate length
blank phone	None	Invalid/missing phone remains null
"1,200.50"	1200.50	Remove grouping comma; Decimal
■ 1500	1500.00	Remove currency symbol
Mohammed, Irfan	Mohammed, Irfan	Preserve comma; title-case words
blank name	None	Do not fabricate identity

The utility functions return None for invalid or missing input rather than crashing. The date parser intentionally accepts the declared list only; a production version should take source-specific formats from configuration and reject ambiguous strings when the source convention is not established.

Task 4: Comparing Data-Wrangling Tools

Scores are qualitative design assessments on a 1–5 scale (5 is strongest for CityCare). They are not benchmark measurements. CityCare has two analysts, no software budget and a strict privacy requirement, so offline, reproducible and low-cost operation receives high weight.

Tool	Learning	Scale	Automation	CSV/JSON/XML	Audit / repeat	Cost & offline	Overall
Python standard library	3	3	5	5	5	5	26/30
pandas	3	4	5	4	4	5	25/30
Spreadsheet	5	2	2	3	2	4	18/30
OpenRefine	4	3	3	4	3	5	22/30
SQLite / SQL	3	4	5	2	5	5	24/30
Commercial ETL (Alteryx/Trifacta)	3	5	5	5	4	1	23/30

The totals are simple unweighted sums across six criteria; a formal procurement decision should apply explicit weights. Python standard library scores strongly because it is free, offline-capable and supports CSV, JSON, XML, SQLite, logging and unittest without external dependencies. pandas would be faster for larger tabular transformations but adds a dependency and does not replace careful nested JSON/XML parsing.

4.1 Mini-experiment design

Use a fixed list of at least 100 date strings containing ISO, DD/MM/YYYY, DD-MM-YYYY and deliberately invalid values. Run the same parsing rule in Python and a spreadsheet tool. Record five Python runs with time.perf_counter(), and record spreadsheet import/transform time manually. The assignment seed contains only six CSV records, so it is not sufficient evidence for a 100-value benchmark.

```
import time
from datetime import datetime

samples = (["2026-10-03", "03/10/2026", "03-10-2026",
            "not-a-date"] * 25)
formats = ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y")

def parse(s):
    for fmt in formats:
        try: return datetime.strptime(s, fmt).date()
        except ValueError: continue
    return None

times = []
for _ in range(5):
    start = time.perf_counter()
    results = [parse(x) for x in samples]
    times.append(time.perf_counter() - start)
print("unparseable:", sum(x is None for x in results))
print("average seconds:", sum(times) / len(times))
```

Expected correctness for this constructed list: 25 of 100 values are unparseable because one quarter are the deliberate invalid string. Actual elapsed time depends on the machine and must be copied from execution; it is not fabricated here. Spreadsheet reproducibility requires saving the import locale, transformation steps and workbook. A script is more reliably repeatable in scheduled runs.

4.2 Recommendation and limitations

• Use Python standard library as the main orchestration and transformation layer, SQLite as the constrained local target, and optionally OpenRefine for supervised exploration of unusual values.

• Limitation 1: standard-library code requires more development and testing than a visual ETL product.

• Limitation 2: a single-process Python pipeline may need streaming, batching and database indexing as volume grows.

• Limitation 3: manual exception review is still required for ambiguous patient identity and clinical values.

Task 5: Python Foundations – Generating and Profiling Dirty Data

The generator must use a fixed seed so that the same seed creates the same defects and linkage pattern. The required minimums are 200 CSV visits (at least 150 unique patients), 100 JSON reports (average at least two tests) and 100 XML dispensings (1–5 items). Inject at least 5% duplicate CSV rows, missing values in three fields at 8% or more, three date formats, three gender spellings, two patient-ID styles, five malformed records per file, and ensure at least 60% of lab/dispensing records link to patients in Source A.

5.1 Reproducible generator pattern

```
# generate_data.py – core deterministic pattern
import argparse, csv, json, random
from pathlib import Path
import xml.etree.ElementTree as ET

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", default="generated")
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()
    random.seed(args.seed)
    out = Path(args.out); out.mkdir(parents=True, exist_ok=True)

    patients = [f"P{1000+i}" for i in range(160)]
    visits = []
    genders = ["M", "Male", "m", "F", "Female"]
    dates = ["2026-10-03", "03/10/2026", "03-10-2026"]
    for i in range(190):
        visits.append([random.choice(patients), f"Patient {i}",
            random.choice(genders), "", "", random.choice(dates),
            random.choice(["ENT","Cardiology","General Medicine"]),
            str(random.choice([600,850,1200.50]))])
    visits += visits[:10]                  # 5% duplicates
    with open(out/"source_a.csv","w",newline="",encoding="utf-8") as f:
        w=csv.writer(f); w.writerow(["patient_id","full_name","gender",
          "dob","phone","visit_date","department","fee_paid"]); w.writerows(visits)
    reports=[]
    for i in range(100):
        pid=random.choice(patients) if i < 70 else "P99999"
        reports.append({"lab_report_id":f"LR-{i:05d}",
          "patient_ref":pid,"tests":[
          {"code":"HB","value":13.2,"unit":"g/dL"},
          {"code":"GLU-F","value":"112","unit":"mg/dL"}]})
    (out/"source_b.json").write_text(json.dumps(reports, indent=2),encoding="utf-8")
    root=ET.Element("pharmacy_dispensing")
    for i in range(100):
        d=ET.SubElement(root,"dispense",id=f"RX-{i:05d}",date="2026-10-03")
        ET.SubElement(d,"patient",id=random.choice(patients))
        items=ET.SubElement(d,"items")
        for j in range(random.randint(1,5)):
            ET.SubElement(items,"item",drug_code=f"DR{j}",
              qty="2",unit_price="3.50").text="Sample medicine"
    ET.ElementTree(root).write(out/"source_c.xml",encoding="utf-8",
                               xml_declaration=True)
    print("Generated deterministic seed:", args.seed)
if __name__ == "__main__": main()
```

This is a compact starter generator, not the full assessed generator: the student must extend it to inject the required missingness rates, five malformed records in each format, varying ID styles, and the required orphan/linkage proportions, then verify achieved rates in profile.py. It deliberately does not claim those requirements have already been met.

5.2 Profiling and injection log

Source	Required profile output	Target injection	Verification
A CSV	Rows, unique patient IDs, missing per column, gender/department spellings	≥5% duplicates; ≥8% missing in three fields; ≥3 date/gender styles; ≥5 malformed rows	Compare injected count with measured count
B JSON	Report count, key paths/types, tests per report, patient linkage	≥5 malformed records; mixed IDs/types; ≥60% linked	Count parse failures, missing blocks and orphans
C XML	Dispense count, item count, invalid quantities, missing totals	≥5 malformed records; 1–5 items; ≥60% linked	Count ParseError files/records and item defects

A profile report should calculate achieved rate = observed defect count / eligible record count × 100. The injection log should include defect, target rate, injected count, observed count, achieved rate and pass/fail. Never report target values as measured results.

Task 6: Machine-Readable versus Human-Readable Data

CityCare artefact	Classification	Reason
Scanned handwritten prescription	Human-readable only (unless OCR applied)	Pixels contain no guaranteed text fields or schema
Printed bill PDF	Both, if text-based; human-readable layout plus extractable text	Machine extraction may still need layout rules
OPD CSV export	Both	Delimited rows and quoting rules; humans can inspect values
Nested lab JSON	Both	Explicit keys, arrays and JSON syntax
Pharmacy XML	Both	Tags/attributes define structure; text is readable
PDF lab report scan	Human-readable only	Image-only scan lacks embedded machine text
Free-text discharge summary	Human-readable; partially machine-readable	Text can be tokenised but fields and meanings are not guaranteed
Relational database table	Machine-readable and human-readable with a query tool	Typed columns and schema; values can be displayed
CSV in spreadsheet	Both	Spreadsheet renders cells, but locale/type inference can alter data

6.1 Readable to a person, ambiguous to a program

First, the CSV value "Mohammed, Irfan" is one full_name field because it is quoted. A naive comma split produces two columns and shifts every later value. A standards-aware csv.DictReader with quoting support correctly treats the comma as part of the name. Second, "■ 1500" looks obvious to a person but is not a plain numeric token. The rupee symbol may also depend on UTF-8 encoding. A parser must decode explicitly, remove a recognised currency marker and parse a Decimal; blindly calling float() fails or produces inconsistent values.

6.2 Format rules

Format	Syntax / well-formedness	Nesting / types / comments	Schema support	Ill-formed example
CSV	Rows, delimiter, quoting and escaping; quoted field can contain comma/newline	Flat rows; values are text until converted; no standard comments	Header and external dialect/schema conventions	Unclosed quote: `a,b\n1,"x`
JSON	Objects use braces, arrays brackets, strings double quotes, commas and colons required	Nested; string/number/boolean/null; no comments in standard JSON	JSON Schema can validate structure	`{"id": 1,}` trailing comma
XML	One root; matching case-sensitive tags; properly nested; quoted attributes; escaped reserved chars	Nested elements; attributes for metadata; text for content; comments allowed	DTD/XSD can define structure and types	`<root><item></root>` mismatched tags

6.3 Well-formedness demonstration

```
import csv, json
import xml.etree.ElementTree as ET

tests = [
    ("CSV", lambda: list(csv.reader(['a,b', '1,"x'']))),
    ("JSON", lambda: json.loads('{"id": 1,}')),
    ("XML", lambda: ET.fromstring("<root><item></root>"))
]
for name, parse in tests:
    try: parse()
    except Exception as exc:
        print(name, type(exc).__name__, str(exc))
```

The CSV module is permissive by default and may accept an unterminated quoted field rather than raise an exception. To demonstrate strict CSV handling, construct csv.reader(..., strict=True) and catch csv.Error. JSON raises JSONDecodeError for the trailing comma. ElementTree raises ParseError for mismatched tags. Exact error wording varies by Python version.

Task 7: Reading, Analysing and Writing CSV Data

Read with utf-8-sig so a UTF-8 byte-order mark does not become part of the first header. Use newline='' as required by the csv module. DictReader handles quoted commas. Blank lines should be skipped; rows with extra or missing fields should be routed to a rejected file with a reason.

```
# csv_wrangler.py – core processing pattern
import csv
from collections import defaultdict
from utils import normalise_gender, parse_date, clean_phone, parse_amount, standardise_name

INPUT = "source_a.csv"
CLEAN = "clean_visits.csv"
REJECT = "rejected_visits.csv"
FIELDS = ["patient_id","full_name","gender","dob","phone",
          "visit_date","department","fee_amount"]
seen, clean, rejected, duplicates = set(), [], [], 0
with open(INPUT, newline="", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for line, row in enumerate(reader, start=2):
        if not row or not any((v or "").strip() for v in row.values()):
            continue
        pid = (row.get("patient_id") or "").strip().upper()
        date = parse_date(row.get("visit_date"))
        dept = (row.get("department") or "").strip()
        name = standardise_name(row.get("full_name"))
        fee = parse_amount(row.get("fee_paid"))
        if not pid or not date or not dept or not name:
            rejected.append({"line":line,"reason":"REQUIRED_OR_INVALID"})
            continue
        key=(pid,date,dept)
        if key in seen:
            duplicates += 1; continue
        seen.add(key)
        clean.append(dict(patient_id=pid, full_name=name,
          gender=normalise_gender(row.get("gender")) or "U",
          dob=parse_date(row.get("dob")), phone=clean_phone(row.get("phone")),
          visit_date=date, department=dept, fee_amount=str(fee) if fee is not None else ""))
with open(CLEAN,"w",newline="",encoding="utf-8") as f:
    w=csv.DictWriter(f,fieldnames=FIELDS); w.writeheader(); w.writerows(clean)
with open(REJECT,"w",newline="",encoding="utf-8") as f:
    w=csv.DictWriter(f,fieldnames=["line","reason"]); w.writeheader(); w.writerows(rejected)
print("clean",len(clean),"rejected",len(rejected),"duplicates",duplicates)
```

For the six-row sample, the repeated P1001 visit is a duplicate under the required key. P1005 is invalid for patient-master insertion because full_name is blank. P1006 has a missing DOB and fee NA; the visit may be retained if the policy permits nullable DOB and fee, but fee is flagged for billing. P1002's missing phone is retained with a flag. The exact clean/rejected counts depend on whether the implementation rejects incomplete patient names before or after staging; this report's rule rejects the P1005 row from the clean visit output.

7.1 Reconciliation and aggregation

The reconciliation equation is input nonblank data rows = clean rows + rejected rows + duplicates removed. If blank physical lines are excluded, report them separately so the equation remains explicit. Department fee totals use Decimal and ignore null fees while separately counting fee exceptions.

```
from decimal import Decimal
totals, counts = defaultdict(Decimal), defaultdict(int)
for row in clean:
    if row["fee_amount"]:
        totals[row["department"]] += Decimal(row["fee_amount"])
        counts[row["department"]] += 1
for dept in sorted(totals):
    print(dept, totals[dept], "paid visits:", counts[dept])
```

CSV pitfall: manual splitting on commas corrupts quoted names and currency fields. The csv module is used with explicit encoding and quoting support. A second pitfall is automatic spreadsheet conversion of identifiers or dates; IDs are treated as strings.

Task 8: Reading, Analysing and Writing JSON Data

8.1 Structure inventory

JSON key path	Observed type	Presence / handling
lab_report_id	str	Required report identifier
patient_ref	str	Required for patient matching
collected_at	str	Required timestamp with offset
ordered_by	dict or absent	Optional block; use get()
ordered_by.doctor_id	str	Optional if ordered_by absent
ordered_by.name	str	Optional
ordered_by.department	str	Optional
tests	list[dict]	Required list; one report may have multiple tests
tests[].code / name / unit	str	Expected test metadata
tests[].value	float / numeric str / status str	Convert numeric values; pending remains status
tests[].ref_range	dict or null	Optional range
tests[].ref_range.low/high	int/float or absent	Use only when both numeric bounds exist
status	str	Report workflow state, e.g. FINAL / IN_PROGRESS

8.2 Safe flattening and flags

```
# json_wrangler.py – core flattening logic
import json
from datetime import datetime
from pathlib import Path

def canonical_id(value):
    s=str(value or "").strip().upper()
    s=s.removeprefix("PT-").removeprefix("P")
    return "P"+s if s.isdigit() else None

def flatten_report(report):
    rows, errors = [], []
    rid=report.get("lab_report_id")
    pid=canonical_id(report.get("patient_ref"))
    if not rid or not pid:
        return [], [{"report_id":rid,"reason":"MISSING_ID"}]
    doctor=report.get("ordered_by") or {}
    for test in report.get("tests") or []:
        raw=test.get("value")
        try: value=float(raw)
        except (TypeError, ValueError):
            value=None
        flag="UNKNOWN"
        rr=test.get("ref_range")
        if value is not None and isinstance(rr,dict):
            low, high=rr.get("low"), rr.get("high")
            if isinstance(low,(int,float)) and isinstance(high,(int,float)):
                flag="LOW" if value<low else "HIGH" if value>high else "NORMAL"
        rows.append({"report_id":rid,"patient_id":pid,
          "collected_date":(report.get("collected_at") or "")[:10],
          "doctor_id":doctor.get("doctor_id"),"test_code":test.get("code"),
          "value_num":value,"unit":test.get("unit"),"flag":flag,
          "state":"PENDING" if str(raw).lower()=="pending" else "MEASURED"})
    return rows, errors

# JSON text and file loading examples
obj=json.loads('[{"id": 1}]')
with open("source_b.json",encoding="utf-8") as f:
    reports=json.load(f)
```

In the seed data, report LR-20261003-0042 has patient_ref 1001, which canonicalises to P1001. Haemoglobin 13.2 is within 13.0–17.0 and is NORMAL. Fasting glucose is numeric string 112 but its reference range is null, so its flag is UNKNOWN. LR-20261003-0057 has patient P1002 and a TSH value “pending”; it is not a numeric result and remains PENDING/UNKNOWN. The absent ordered_by block must not crash processing.

8.3 Example flattened records and exceptions

Report	Patient	Test	Numeric value	Flag / state
LR-20261003-0042	P1001	HB	13.2	NORMAL / MEASURED

Report	Patient	Test	Numeric value	Flag / state
LR-20261003-0042	P1001	GLU-F	112	UNKNOWN / MEASURED
LR-20261003-0057	P1002	TSH	null	UNKNOWN / PENDING

No exception is required for the two sample reports if the required report ID, patient reference and tests are present. An exception is created for a missing/invalid patient reference, malformed timestamp, absent test code, or a test object that cannot be flattened. Exception examples should be generated by the negative tests rather than invented as observed seed failures.

8.4 Flag rates

For each test code, calculate LOW/NORMAL/HIGH/UNKNOWN counts divided by the number of flattened records for that code. Do not calculate a rate from the two sample records and present it as a population estimate; the supplied seed is illustrative only.

Task 9: Reading, Analysing and Writing XML Data

9.1 XML navigation and representation

Use ElementTree.parse() to load the document, root.findall('dispense') to iterate records, dispense.get('id') for the dispense identifier, dispense.get('date') for its date, find('patient') for patient attributes, findall('items/item') for line items, and find('total') for the stated total. Attributes are suitable for compact identifiers and metadata (dispense id, date, patient id); child elements are suitable for repeatable or descriptive content (items, medicine name, total).

9.2 Validation and processing

```
# xml_wrangler.py – core extraction and total check
import xml.etree.ElementTree as ET
from datetime import datetime
from decimal import Decimal, InvalidOperation

def parse_xml_date(s):
    for fmt in ("%Y-%m-%d", "%d.%m.%Y"):
        try: return datetime.strptime(s,fmt).date().isoformat()
        except (ValueError, TypeError): pass
    return None

tree=ET.parse("source_c.xml")
root=tree.getroot()
issues=[]; flat=[]
for d in root.findall("dispense"):
    rid=d.get("id")
    date=parse_xml_date(d.get("date"))
    patient=d.find("patient")
    items=d.findall("items/item")
    computed=Decimal("0.00"); valid=True
    for item in items:
        try:
            qty=int(item.get("qty"))
            price=Decimal(item.get("unit_price"))
            if qty < 0 or price < 0: raise ValueError()
            computed += qty*price
            flat.append([rid, patient.get("id") if patient is not None else "",
                         item.get("drug_code"), item.text or "", qty, str(price)])
        except (TypeError, ValueError, InvalidOperation):
            valid=False; issues.append((rid,"INVALID_ITEM"))
    total=d.findtext("total")
    if total is None:
        issues.append((rid,"MISSING_TOTAL"))
    elif valid and abs(Decimal(total)-computed) > Decimal("0.01"):
        issues.append((rid,"TOTAL_MISMATCH"))
```

RX-88121: computed total is INR 90.00 and matches the stated total exactly. RX-88122: qty="two" is invalid and total is missing; therefore no reliable computed total is available. It is incorrect to claim a numeric discrepancy for RX-88122 because no valid computed amount exists.

9.3 Output design

Element / attribute	Purpose
dispensing_clean (root)	Document root; source branch attribute
dispense @id	Unique dispensing identifier
dispense @date	ISO 8601 local dispensing date
patient @id	Canonical patient identifier
item @drug_code	Medicine code
item @quantity	Validated integer quantity
item @unit_price	Decimal price in INR
item name	Human-readable medicine name
total	Recomputed amount and currency attribute

```
# Safe parse failure
try:
    ET.parse("malformed.xml")
except ET.ParseError as exc:
    print("Malformed XML:", exc)
    print("Line/column:", exc.position)
```

Use ET.indent(tree, space=' ') before writing the clean XML. Write one CSV row per item using csv.DictWriter. An empty items block is retained as an exception because a dispensing record without items cannot support a valid total. The largest discrepancy is computed only across records with valid numeric items and a stated total.

Task 10: Understanding and Implementing the Database Schema

Table	Columns and types	Keys / nullability
DEPARTMENT	department_id INT; department_name VARCHAR(50)	PK department_id; name UNIQUE; name NOT NULL
DOCTOR	doctor_id INT; doctor_name VARCHAR(80); department_id INT	PK doctor_id; department_id FK -> DEPARTMENT; names/department NOT NULL
PATIENT	patient_id INT; full_name VARCHAR(80); gender CHAR(1); date_of_birth DATE; phone VARCHAR(15); registered_on DATE	PK patient_id; full_name NOT NULL; other nullability not specified
VISIT	visit_id INT; patient_id INT; doctor_id INT; visit_date DATE; fee_amount DECIMAL(8,2)	PK visit_id; patient_id FK -> PATIENT; doctor_id FK -> DOCTOR; nullability not fully specified
LAB_REPORT	report_id VARCHAR(20); visit_id INT; collected_at DATETIME; status VARCHAR(12)	PK report_id; visit_id FK -> VISIT
LAB_RESULT	result_id INT; report_id VARCHAR(20); test_code VARCHAR(10); result_value VARCHAR(20); unit VARCHAR(10)	PK result_id; report_id FK -> LAB_REPORT
PRESCRIPTION	rx_id VARCHAR(12); visit_id INT; dispensed_on DATE; total_amount DECIMAL(8,2)	PK rx_id; visit_id FK -> VISIT
RX_ITEM	rx_id VARCHAR(12); drug_code VARCHAR(10); quantity INT; unit_price DECIMAL(6,2)	Composite PK (rx_id, drug_code); rx_id FK -> PRESCRIPTION

The brief explicitly states NOT NULL only for PATIENT.full_name and intends DEPARTMENT.department_name to be unique. Other nullability is not documented; the improved design below makes operationally required keys and dates NOT NULL. The source schema table name appears split as “PRESCRIPTIO N”; it is interpreted as PRESCRIPTION.

10.1 ER relationships

```
DEPARTMENT 1 ---- many DOCTOR
PATIENT    1 ---- many VISIT
DOCTOR     1 ---- many VISIT
VISIT      1 ---- 0..many LAB_REPORT
LAB_REPORT 1 ---- 0..many LAB_RESULT
VISIT      1 ---- 0..many PRESCRIPTION
PRESCRIPTION 1 --- 1..many RX_ITEM
RX_ITEM primary key = (rx_id, drug_code)
```

10.2 Schema weaknesses and corrections

Weakness	Impact	Correction
PATIENT.patient_id is INT but sources contain P1001 / PT- identifiers	IDs may be truncated, coerced or fail joins	Use VARCHAR(12) canonical ID; maintain alias table for legacy identifiers
No CHECK constraint for gender	Uncontrolled values contaminate reports	CHECK gender IN ('M','F','U')
LAB_RESULT.result_value is VARCHAR(20)	Numeric and status text mixed; hard to aggregate safely	Separate value_num DECIMAL, result_text and result_state
PRESCRIPTION has no prescriber/doctor link	Cannot trace prescribing responsibility	Add doctor_id FK and source provenance
RX_ITEM quantity has no positivity check	Negative or zero quantities accepted	CHECK quantity > 0
No uniqueness for source visit identity	Duplicate visits may load	UNIQUE(patient_id, visit_date, department_id) where business rules permit
No audit columns	Changes not traceable	Add source_system, source_record_id, created_at, run_id

10.3 Improved SQLite DDL

```
PRAGMA foreign_keys = ON;
CREATE TABLE department(
 department_id INTEGER PRIMARY KEY,
 department_name TEXT NOT NULL UNIQUE
);
CREATE TABLE doctor(
 doctor_id INTEGER PRIMARY KEY,
 doctor_name TEXT NOT NULL,
 department_id INTEGER NOT NULL REFERENCES department(department_id)
);
CREATE TABLE patient(
 patient_id TEXT PRIMARY KEY,
 full_name TEXT NOT NULL,
 gender TEXT NOT NULL CHECK(gender IN ('M','F','U')),
 date_of_birth TEXT,
 phone TEXT,
```

```
 registered_on TEXT
);
CREATE TABLE visit(
 visit_id INTEGER PRIMARY KEY,
 patient_id TEXT NOT NULL REFERENCES patient(patient_id),
 doctor_id INTEGER REFERENCES doctor(doctor_id),
 department_id INTEGER NOT NULL REFERENCES department(department_id),
 visit_date TEXT NOT NULL,
 fee_amount NUMERIC CHECK(fee_amount IS NULL OR fee_amount >= 0),
 UNIQUE(patient_id, visit_date, department_id)
);
CREATE TABLE lab_report(
 report_id TEXT PRIMARY KEY,
 visit_id INTEGER NOT NULL REFERENCES visit(visit_id),
 collected_at TEXT NOT NULL,
 status TEXT NOT NULL
);
CREATE TABLE lab_result(
 result_id INTEGER PRIMARY KEY,
 report_id TEXT NOT NULL REFERENCES lab_report(report_id),
 test_code TEXT NOT NULL,
 value_num NUMERIC,
 result_text TEXT,
 result_state TEXT NOT NULL,
 unit TEXT
);
CREATE TABLE prescription(
 rx_id TEXT PRIMARY KEY,
 visit_id INTEGER NOT NULL REFERENCES visit(visit_id),
 dispensed_on TEXT NOT NULL,
 total_amount NUMERIC CHECK(total_amount IS NULL OR total_amount >= 0)
);
CREATE TABLE rx_item(
 rx_id TEXT NOT NULL REFERENCES prescription(rx_id),
 drug_code TEXT NOT NULL,
 quantity INTEGER NOT NULL CHECK(quantity > 0),
 unit_price NUMERIC NOT NULL CHECK(unit_price >= 0),
 PRIMARY KEY(rx_id, drug_code)
);
```

Constraint demonstration should insert five valid rows and then attempt: duplicate department name, invalid gender, visit with unknown patient, zero quantity and negative fee. Each should raise sqlite3.IntegrityError. Exact error text is SQLite-version dependent; report the message printed by the local run rather than fabricating it.

Task 11: Integrating Multi-Source Data and Converting Between Formats

11.1 Identifier normalisation and matching

For legacy IDs, trim whitespace, uppercase, remove an optional P or PT- prefix, verify that the remainder is numeric, then store a canonical legacy key such as P1001. Preserve the original ID in a source-alias table. Do not strip leading zeros from a national identifier. Day-4 national IDs are a separate identifier namespace and must not be converted to legacy numeric IDs.

Raw value	Canonical legacy ID	Decision
1001	P1001	Prefix canonicalised
P1001	P1001	Already canonical
PT-1001	P1001	Legacy prefix canonicalised if source policy confirms
P1006	P1006	Canonicalised
unknown / blank	null	Unresolved; quarantine or manual review

The supplied sample contains one clear lab match (1001 to P1001), one lab match (P1002), and pharmacy references 1003 and P1005 that correspond to CSV P1003 and P1005. RX-88122 has an invalid item quantity, but its patient ID itself can still be resolved. These are sample-level link observations, not counts for the required generated dataset.

11.2 Integration policy

• Match by canonical patient ID first. Link lab reports to a visit on the same patient within a documented window of ±1 day from visit_date; if multiple visits qualify, mark ambiguous rather than choose arbitrarily.

• Link pharmacy dispensing to a visit for the same patient within ±1 day. A prescription with no matching visit is an orphan and is not loaded against a fabricated visit.

• Store matched, orphan and ambiguous counts separately for each source. Maintain source record IDs to make each link reversible.

• Load in dependency order: departments, doctors, patients, visits, lab reports/results, prescriptions/items. Enable PRAGMA foreign_keys=ON and use a transaction.

11.3 Consumer formats and round trip

Output	Structure	Best consumer
patient_summary.json	Nested patient object containing visits, labs and prescriptions	Web/mobile API; preserves hierarchy
visits_flat.csv	One visit per row, fixed columns and ISO dates	Billing spreadsheet and tabular analysis
dispensing_report.xml	Structured dispensing records with attributes and item elements	Archival or partner exchange requiring XML schema validation

CSV → JSON → CSV can preserve cell text and row order if field names and ordering are explicitly retained. It can alter types (numeric strings may become numbers), null versus empty string, leading zeros, date formatting and quoting. JSON adds nesting; flattening it back to CSV requires a defined path-to-column mapping. XML attributes and elements also have different semantics and are not naturally preserved in a flat CSV.

11.4 Format comparison

Criterion	CSV	JSON	XML
File size	Usually smallest for flat data	Moderate; key names repeated	Often largest due to tags
Parse time	Fast for flat rows	Fast for nested objects with standard parser	Usually more verbose; parser overhead
Nesting	No native nesting	Native objects and arrays	Native tree structure
Data typing	Text; external conversion	String, number, boolean, null	Text; schema can type values
Human readability	High for tables	High for structured objects	High but verbose
Schema validation	External dialect/contracts	JSON Schema	DTD / XSD

For billing, use CSV because staff need flat rows and spreadsheet compatibility. For a mobile API, use JSON because it naturally represents patient → visits → tests. For government archival exchange, use XML when the receiving body's schema and

validation requirements specify XML; otherwise follow the mandated format rather than choosing by preference.

Task 12: Testing, Validation and Critical Evaluation

12.1 Test plan

Test ID	Objective	Input	Expected	Actual	Result
U01	M gender normalisation	Male	M	M	Pass
U02	F gender normalisation	female	F	F	Pass
U03	ISO date parsing	2026-10-03	2026-10-03	2026-10-03	Pass
U04	Invalid date safe handling	32/13/2026	None	None	Pass
U05	Amount parsing	■1500	1500.00	1500.00	Pass
U06	Invalid amount	NA	None	None	Pass
C01	CSV quoted comma	Mohammed, Irfan	One name field	One name field	Pass
C02	CSV duplicate key	Repeated P1001	Second row removed	Second row removed	Pass
J01	JSON pending value	pending	Null numeric + PENDING	Null numeric + PENDING	Pass
J02	JSON absent doctor block	No ordered_by	No crash; doctor null	No crash; doctor null	Pass
X01	XML valid total	RX-88121	90.00 match	90.00 match	Pass
X02	XML invalid quantity	two	Exception recorded	Exception recorded	Pass
I01	Canonical ID join	1001 and P1001	Same canonical ID	P1001	Pass
I02	Orphan patient	P99999	No visit FK load	Quarantine	Pass
E01	Malformed JSON	Trailing comma	JSONDecodeError caught	Caught by handler	Pass
E02	Malformed XML	Mismatched tag	ParseError with position	Caught by handler	Pass
E03	Foreign-key violation	Unknown patient visit	IntegrityError	Rejected	Pass

The table is a proposed test specification with expected outcomes; “Actual” values describe deterministic function-level behaviour, not an execution transcript from a local run. Replace or confirm them with the actual unittest output before submission. A complete implementation should assert each expected value and capture the test runner's result.

12.2 Quality metrics

Metric	Formula	Before (seed observation)	After target / acceptance
Completeness	Non-missing required field cells / all required field cells × 100	Not fully quantifiable across all sources from the small sample	≥99% for required fields; optional missing values explicitly flagged
Validity	Values passing field format/domain / values checked × 100	Mixed dates, gender and amount strings visibly fail canonical format	≥99% valid visit dates; invalids quarantined
Uniqueness	Duplicate business keys / visit rows × 100	One repeated P1001 visit in six sample rows	0 duplicate business keys in curated visits
Consistency	Cross-source IDs matched / eligible IDs × 100	Two lab refs and two pharmacy refs appear resolvable in sample; no full denominator	≥99% among eligible, non-orphan records
Accuracy proxy	Valid stated totals matching recomputed totals / valid comparable totals × 100	RX-88121 matches; RX-88122 not comparable due invalid qty and missing total	100% of loaded comparable totals within INR 0.01

12.3 Critical evaluation

The pipeline improves trust by separating raw, staged and curated data; using deterministic parsing rules; retaining exceptions; and loading only records that pass database constraints. However, several residual risks remain. First, dates such as 03-10-2026 are ambiguous without source-specific locale documentation. The sample context supports interpreting the OPD value as 3 October 2026, but this assumption must be confirmed with the source owner before production. Second, identifier normalisation can create false matches if two systems reuse the same numeric suffix. A canonical ID rule is therefore not sufficient by itself: alias tables, source namespaces and manual review of collisions are required. Third, missing DOB, phone, lab reference ranges and pharmacy totals cannot always be safely imputed. The pipeline should preserve nulls and flags rather than manufacture values. Orphan records may remain until a registration or identity team resolves them. A scalability concern is loading every JSON report or a large XML document into memory at once. The pipeline should process records in batches, stream JSON Lines, use incremental XML parsing where appropriate, create indexes on patient IDs and visit dates, and commit database transactions in

controlled batches. Privacy and ethics are equally important. Patient names, phone numbers and clinical results are sensitive. Access should be role-based, files encrypted at rest and in transit, logs masked, and shared analytical extracts pseudonymised. Retention periods and audit access should be approved by the hospital. The pipeline's quality metrics are proxies, not proof of clinical accuracy: only source reconciliation and authorised clinical review can establish whether a result is correct for a patient.

Day-4 Design Challenge: New Telemedicine Source D

The new source is JSON Lines with 20,000 sessions per day and backlog files above 250 MB. It introduces national IDs, optional legacy IDs, UTC timestamps, European decimal fees, missing doctor blocks, null quantities and null fees. The original architecture is reusable, but its file reader, identifier model, time handling, schema and integration rules require modification.

D1. Impact analysis

Component	Impact	Decision	Cause / predicted behaviour
Workflow stages	Minor	MODIFY	Keep stages; add streaming ingest, timezone conversion and Source-D quarantine branch
Utility functions	Major	MODIFY	Add national-ID validation, UTC conversion, European Decimal parser and duration parser
CSV program	None	RETAIN	Source A unchanged; add regression tests
JSON program	Major	MODIFY	JSON Lines is one object per line, not one JSON array; json.load on whole file is unsuitable
XML program	None	RETAIN	Source C unchanged; preserve its own tests
Database schema/loading	Breaking	REDESIGN	Patient key changes INT to VARCHAR national ID; add TELE_SESSION and VISIT.visit_mode
Integration logic	Breaking	REDESIGN	Legacy alias may be null; national IDs must not be conflated with local IDs
Tests/metrics	Major	MODIFY	Add 20k streaming, null fee, absent doctor, malformed-line and timezone tests

Prediction if the original pipeline is run unchanged: it may reject the entire JSON Lines file because the root is not a JSON array; if it loads lines individually without changes, national IDs may fail integer-oriented ID logic, UTC timestamps may be treated as local time, and “1.250,00 INR” may fail the original amount parser. Null legacy IDs and absent doctor blocks may cause unmatched records or key errors. These are predictions from the stated design constraints; actual crash logs and observed row counts must be captured by running the original program against the generated Day-4 file.

D2. Evaluation of original design

Extensibility is helped by separating utility functions, centralising paths and thresholds in config.py, keeping source-specific parsers independent, using logging, and enforcing relational constraints. Those choices allow a new parser and migration to be added without rewriting CSV or XML handling. Extensibility is hurt if the original code assumes a single JSON document, loads all records with json.load, assumes patient IDs are integers, hard-codes local timestamps, or assumes US-style decimal points. The 20,000-session and 250 MB backlog requirements make memory behaviour a design criterion. Compare json.load with line-by-line parsing using time.perf_counter() and tracemalloc; record elapsed seconds and peak traced memory for the same file. Do not insert unmeasured benchmark values into this report.

```
# benchmark memory/time pattern
import time, tracemalloc, json

def measure_jsonl(path):
    tracemalloc.start()
    start=time.perf_counter()
    count=0
    with open(path,encoding="utf-8") as f:
        for line in f:
            try: json.loads(line); count += 1
            except json.JSONDecodeError: pass
    elapsed=time.perf_counter()-start
    _, peak=tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return count, elapsed, peak
```

D3. Adaptation: streaming JSON Lines

```
# json_lines_wrangler.py – core streaming parser
import json
from datetime import datetime, timezone, timedelta
from decimal import Decimal, InvalidOperation

LOCAL = timezone(timedelta(hours=5, minutes=30))

def fee_european(raw):
```

```
    if raw is None: return None
    s=str(raw).replace("INR","").strip()
    s=s.replace(".","").replace(",",".")
    try: return Decimal(s).quantize(Decimal("0.01"))
    except InvalidOperation: return None

def local_time(raw):
    dt=datetime.fromisoformat(raw.replace("Z","+00:00"))
    return dt.astimezone(LOCAL).isoformat()

def process(path, output, rejects):
    good=bad=0
    with open(path,encoding="utf-8") as src, \\
         open(output,"w",encoding="utf-8") as dst, \\
         open(rejects,"w",encoding="utf-8") as err:
        for line_no,line in enumerate(src,1):
            try:
                obj=json.loads(line)
                patient=obj.get("patient") or {}
                national=patient.get("national_id")
                if not national or not str(national).startswith("PT-"):
                    raise ValueError("INVALID_NATIONAL_ID")
                row={"session_id":obj["session_id"],
                     "patient_id":national,
                     "legacy_id":patient.get("legacy_id"),
                     "start_local":local_time(obj["start_time"]),
                     "duration_min":int(obj["duration_min"]),
                     "fee":str(fee_european(obj.get("consultation_fee")))
                           if obj.get("consultation_fee") is not None else None,
                     "doctor_id":(obj.get("doctor") or {}).get("doctor_id")}
                dst.write(json.dumps(row,ensure_ascii=False)+"\\n")
                good += 1
            except Exception as exc:
                err.write(json.dumps({"line":line_no,"reason":str(exc)})+"\\n")
                bad += 1
    print("accepted",good,"rejected",bad)
```

For the sample first session, 2026-11-09T14:30:00Z converts to 2026-11-09 20:00:00+05:30; fee “1.250,00 INR” becomes Decimal 1250.00; duration becomes integer 18; legacy_id remains null. For the second, 15:05 UTC becomes 20:35 local; the fee remains null and the legacy ID P1006 is preserved as an alias. Null prescribed quantities should be retained as missing/exception item values, not changed to zero.

D3.1 Revised schema and migration

```
PRAGMA foreign_keys = ON;
-- New national-ID patient key
CREATE TABLE patient_new(
 patient_id TEXT PRIMARY KEY CHECK(length(patient_id) <= 12),
 full_name TEXT NOT NULL,
 gender TEXT CHECK(gender IN ('M','F','U')),
 date_of_birth TEXT, phone TEXT, registered_on TEXT
);
CREATE TABLE patient_alias(
 source_system TEXT NOT NULL,
 source_patient_id TEXT NOT NULL,
 patient_id TEXT NOT NULL REFERENCES patient_new(patient_id),
 PRIMARY KEY(source_system, source_patient_id)
);
ALTER TABLE visit ADD COLUMN visit_mode TEXT NOT NULL DEFAULT 'IN_PERSON'
 CHECK(visit_mode IN ('IN_PERSON','TELE'));
CREATE TABLE tele_session(
 session_id TEXT PRIMARY KEY,
 patient_id TEXT NOT NULL REFERENCES patient_new(patient_id),
 start_time_local TEXT NOT NULL,
 duration_min INTEGER CHECK(duration_min > 0),
 consultation_fee NUMERIC CHECK(consultation_fee IS NULL OR consultation_fee >= 0),
 doctor_id INTEGER REFERENCES doctor(doctor_id),
 legacy_id TEXT
);
```

SQLite cannot directly alter an existing primary-key column type. Migration plan: create patient_new and patient_alias; map every old integer ID to the approved national ID using a verified crosswalk; insert patients into patient_new; update child foreign-key tables through a staging copy or rebuild; validate row counts and foreign keys; switch table names inside a controlled maintenance transaction; retain a backup and rollback script. Do not fabricate national IDs for legacy-only patients—place them in a migration exception queue until an authorised mapping is available.

D4. Evidence-based defence

Component	Decision	Reason	Evidence to capture	Residual risk
Workflow	Modify	Same lifecycle; streaming branch	20k run counts and logs	Partial files / retries
Utilities	Modify	New formats and timezone	Unit tests for fee/time/IDs	Locale assumptions
CSV	Retain	Source A unchanged	Existing CSV regression tests	Source changes

Component	Decision	Reason	Evidence to capture	Residual risk
JSON	Modify	JSON Lines differs from JSON array	Malformed line isolation test	Deep nesting changes
XML	Retain	No XML source change	XML regression tests	New XML variants
Schema	Redesign	National ID key and tele-session entity	Migration row counts + FK check	Unmapped legacy patients
Integration	Redesign	Alias and dual-ID resolution	Matched/orphan/ambiguous counts	False identity matches
Tests/metrics	Modify	Volume and new null/malformed cases	15+ tests; time/memory profile	Synthetic data may not reflect production

The recommended approach is to retain the staged architecture while modifying its ingestion and utility layers and redesigning identity storage and telemedicine integration. Streaming JSON Lines is essential because a large backlog may exceed 250 MB; reading the entire file into memory creates unnecessary peak-memory pressure and can fail on constrained machines. Each line is an independent parse unit, so a malformed line can be quarantined without discarding valid neighbouring sessions. UTC timestamps must be converted with timezone-aware datetime objects, not by adding a fixed number of hours to naive timestamps. The European amount parser must remove thousands separators and replace the decimal comma only after validating the expected pattern. The strongest design choices in the original pipeline are source-specific parsers, reusable utilities, central configuration, immutable raw files, logging and database constraints. They reduce the cost of adding Source D. The weakest assumptions are integer-only patient identifiers, whole-document JSON loading, and a single date/number convention. These assumptions are breaking changes because the new partner uses national IDs, JSON Lines, UTC and European numeric formatting. A new patient_alias table keeps CityCare identifiers traceable without pretending that a legacy ID is a national ID. TELE_SESSION separates telemedicine session details from ordinary in-person visits, while visit_mode distinguishes encounter type. A credible alternative is to land each JSON Lines record in a raw SQLite staging table and perform validation and transformation with SQL. This can be better when the team is SQL-skilled, when set-based joins dominate, or when data must be queried incrementally by multiple analysts. It is less attractive for deeply nested JSON, timezone conversion and irregular per-line error handling, which are clearer in Python. pandas is another alternative for tabular transformations but still needs a JSON Lines reader, careful memory strategy and explicit identity rules. Regardless of implementation, production acceptance requires measured throughput, peak memory, exception counts, foreign-key checks and clinical review of ambiguous identity matches. The decisions in this table are design decisions; the evidence column must be populated with actual run logs and measurements before final submission.

Appendix A: Submission run checklist

• Save the seed examples from the brief as source_a.csv, source_b.json and source_c.xml.

• Complete generate_data.py to meet every volume, linkage and defect-injection requirement; run it twice with the same seed and verify identical outputs.

• Run profile.py and preserve its measured counts and injection log.

• Run setup_project.py and environment_check.py; capture console output.

• Run each wrangler and preserve clean/rejected outputs, first lines, logs and reconciliation totals.

• Run schema.py with foreign keys enabled; capture valid inserts and each IntegrityError.

• Run integrate.py and convert.py; verify round-trip equality and export file sizes/timings over five runs.

• Run python -m unittest -v; replace the proposed Actual column in Task 12 with the real test results.

• Generate Source D with 20,000+ sessions and 100 malformed lines; run old and revised pipelines; capture prediction versus observation.

• Benchmark json.load and streaming parsing with elapsed time and peak memory on the same file.

• Review patient privacy, pseudonymise any shared extracts, and submit the PDF plus code ZIP as required.

Appendix B: Important interpretation notes

The original assignment asks for a report of approximately 25–35 pages and a ZIP of all programs, generated data, outputs, logs and tests. This PDF is a report draft answering the written tasks and includes core implementation patterns. It is not a substitute for the required complete executable project or screenshots/measurements. Replace any proposed or expected result with actual local execution evidence. The brief's seed records are small illustrative examples; they cannot establish the required large-scale experiment results.

File naming: DW_A1_192324028_Report.pdf

Student: Answar Vijes **Register number:** 192324028